k-NN Geometric Gap Detection in Software Requirements via Three-Layer Scoring and Soft Cell Aggregation

DiveAI

1 Introduction

Software requirements specifications are routinely incomplete. A healthcare system may specify authentication and audit logging but omit backup and disaster-recovery requirements that every peer system addresses. Detecting such *underspecification* early—before design commits resources—prevents costly rework.

The challenge is that no universal checklist exists: the set of concerns a project *should* address depends on its domain, scale, and quality-attribute priorities. We therefore frame the problem as a **coverage comparison**: given a corpus of peer projects with known requirement types, identify regions of concern that the corpus covers but the target project does not.

We propose a geometric method that represents each requirement as a unit vector in $\mathbb{R}^d$ via a pretrained sentence encoder, then measures coverage deficits via k-nearest-neighbour distances. Three complementary scoring layers—per-point geometric distance, type-restricted distributional distance, and annotation-free soft-count deficit—are fused into a unified gap score controlled by two hyperparameters. The scores are aggregated onto a human-readable (category $\times$ topic) grid for practitioner consumption.

Six baselines validate that every pipeline component contributes measurable value. The strongest result: on adequately-sized projects ($N \geq 50$), the method achieves 0.935 AUROC for detecting completely absent requirement types—matching a ground-truth count oracle that requires human annotation.

2 Problem Definition

2.1 Dataset and Attributes

We use the PROMISE NFR dataset [2]: $N = 621$ natural-language requirement sentences drawn from $M = 15$ software projects. Each requirement r carries two attributes:

1. **Project identifier** $\mathrm{pid}(r) \in \{\mathrm{P}_1, \ldots, \mathrm{P}_{15}\}$. Project sizes range from 12 to 92 requirements; six have $N_j \geq 50$.
2. **Quality-attribute type** $t(r)$ from a taxonomy of $K_t = 12$ classes: Availability, Fault Tolerance, Functional, Legal, Look & Feel, Maintainability, Operability, Performance, Portability, Scalability, Security, Usability.

2.2 What Is a Gap?

Let $P_* = \{r_1, \ldots, r_{N_*}\}$ be a **target project** under analysis, and $\mathcal{D} = \{P_1, \ldots, P_M\}$ a **reference corpus** of training projects. A sentence encoder g maps each requirement to a unit vector $g(r) \in \mathbb{S}^{d-1}$ (the unit hypersphere in $\mathbb{R}^d$, $d = 1024$).

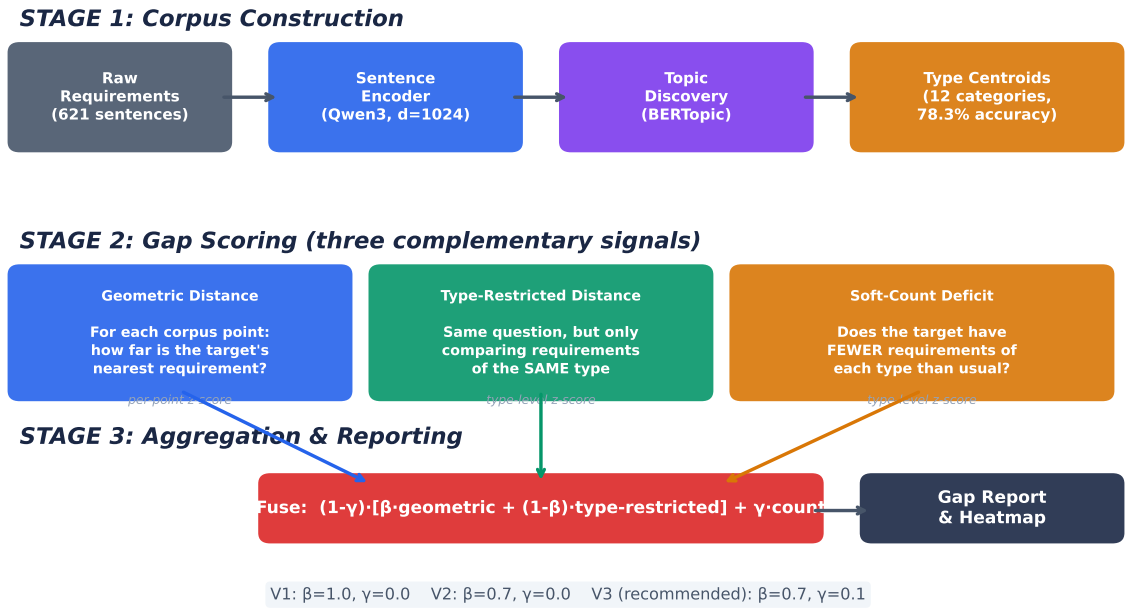

Figure 1: End-to-end pipeline. Stage 1 constructs the reference corpus (embedding, topics, centroids). Stage 2 computes three complementary gap scores, fuses them, aggregates onto a grid, and produces a structured report.

Gap. A region of $\mathbb{S}^{d-1}$ where the reference corpus has substantial coverage (many points from multiple projects) but the target project has little or no coverage (its nearest requirement is far away).

We address three research questions:

RQ1 (Structural gaps) Which requirement types does the *entire corpus* cover but P_* does not?

RQ2 (Domain-specific gaps) Which types do *similar projects* cover but P_* does not?

RQ3 (Novelties) Which requirements in P_* have no analogue in the corpus?

3 Representation: From Text to Geometry

3.1 Sentence Embedding

Every requirement is encoded as a unit vector using Qwen3-Embedding-0.6B [1] ($d = 1024$) in plain passage mode:

$$g(r) \in \mathbb{S}^{d-1}, \qquad \|g(r)\| = 1. \tag{1}$$

Cosine distance is used throughout: $d_{\cos}(u, v) = 1 - u^\top v = \frac{1}{2}\|u - v\|^2$ for unit vectors.

3.2 Category Centroids (Type Axis)

For each of the $K_t = 12$ quality-attribute types, we compute a data centroid from the PROMISE ground-truth labels:

$$\boldsymbol{\mu}_k = \frac{\sum_{r \in \mathcal{R}_k} g(r)}{\left\|\sum_{r \in \mathcal{R}_k} g(r)\right\|}, \qquad h(r) = \arg\min_k d_{\cos}\big(g(r),\, \boldsymbol{\mu}_k\big). \tag{2}$$

Hard assignment $h(r)$ achieves 78.3% accuracy on PROMISE.

3.3 Soft Topic Distributions (Topic Axis)

BERTopic [5] discovers latent topics via UMAP [9] → HDBSCAN [8] → c-TF-IDF. Each requirement receives a **soft distribution** $\boldsymbol{\pi}(r) \in \Delta^{K_s-1}$ via `approximate_distribution()`, ensuring every requirement contributes to every topic with appropriate weight (no outlier loss).

4 Method: Three-Layer Gap Scoring

The method computes a gap score $\Psi[t]$ for each requirement type t. Three layers capture complementary signals; their outputs are fused into a single score controlled by two hyperparameters (β, γ). Figure 1 shows the overall architecture; Figure 2 illustrates what each layer captures.

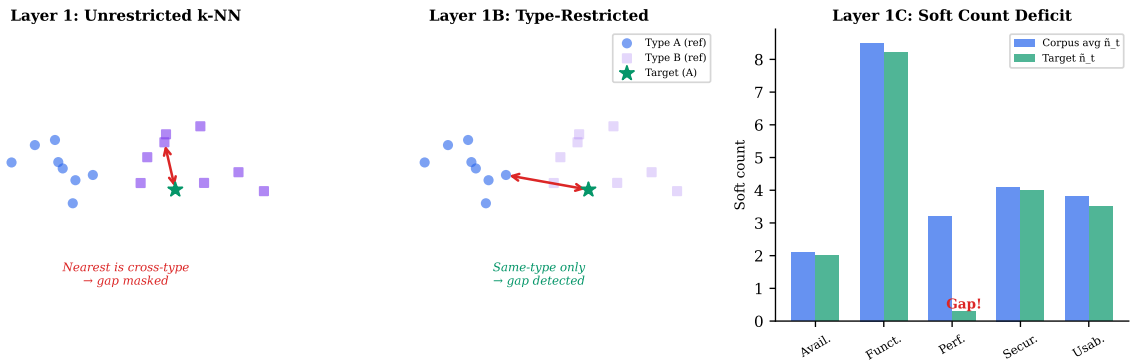

Figure 2: The three scoring layers. **Left:** Layer 1 uses unrestricted k-NN—cross-type neighbours can mask a gap. **Centre:** Layer 1B restricts to same-type neighbours, exposing the gap. **Right:** Layer 1C counts how many soft-assigned requirements of each type the target has vs. the corpus average.

4.1 Layer 1: Per-Point Geometric Gap Score

For each reference point $x \in \mathbf{X}$ (a corpus requirement), we measure whether the target project covers the same region.

Definition 1 (k-NN coverage distance)**.**

$$\varphi(x;\, \mathbf{Y}) = \frac{1}{k} \sum_{i=1}^{k} d_{\cos}\big(x,\, y_{(i)}\big), \tag{3}$$

where $y_{(1)}, \ldots, y_{(k)}$ *are the* k *nearest neighbours of* x *in the target* $\mathbf{Y}$*.*

A large φ means the target has nothing nearby—but this alone is insufficient. Some regions are *inherently* sparse (few projects write portability requirements), so every project is far from those reference points. Thresholding raw φ would flag the same regions for every target, regardless of its actual deficiencies.

Per-project baseline normalisation (the core algorithmic idea). Instead of asking "is φ large?", we ask "is it *unusually* large compared to what a typical individual project achieves?" (Figure 3).

Definition 2 (Per-project baseline)**.** *For each reference point* x*, compute* $\varphi(x; \mathbf{Y}_j)$ *for each of the* M *training projects* individually*:*

$$\varphi_0(x) = \frac{1}{M} \sum_{j=1}^{M} \varphi(x;\, \mathbf{Y}_j), \qquad \hat{\sigma}(x) = \sqrt{\frac{1}{M} \sum_{j=1}^{M} \big(\varphi(x;\, \mathbf{Y}_j) - \varphi_0(x)\big)^2}. \tag{4}$$

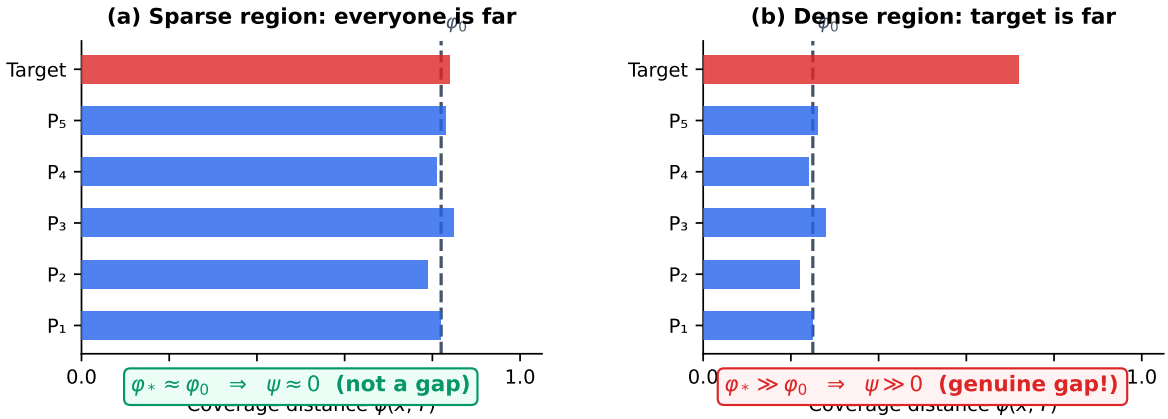

Figure 3: Why per-project normalisation is essential. (a) In a sparse region, all projects are far from x, so $\varphi_* \approx \varphi_0$ and $\psi \approx 0$ (no gap). (b) In a dense region, training projects are close but the target is far, so $\psi \gg 0$ (genuine gap).

Definition 3 (Normalised gap score)**.**

$$\boxed{\psi(x) = \mathrm{clip}\left(\frac{\varphi(x;\, \mathbf{Y}_*) - \varphi_0(x)}{\hat{\sigma}(x) + \varepsilon},\ -5,\ 5\right)} \tag{5}$$

Interpretation: $\psi(x) \gg 0$ means the target is unusually far from x (gap); $\psi(x) \approx 0$ means typical coverage; $\psi(x) \ll 0$ means surplus.

The per-point scores are marginalised to type level via cell aggregation (Section 4.5):

$$\Psi_{\mathrm{unr}}[t] = \frac{\sum_s W[t,s] \cdot \Psi_{\mathrm{cell}}[t,s]}{\sum_s W[t,s]}, \tag{6}$$

where $W[t,s]$ and $\Psi_{\mathrm{cell}}[t,s]$ are the soft-aggregated weights and gap scores (defined in Section 4.5).

Why $k = 1$ for Layer 1. On $\mathbb{S}^{d-1}$ with $d = 1024$, concentration of measure [7] causes all but the 1st nearest neighbour to lie at approximately the same "ambient" distance. Averaging over $k > 1$ adds noise. The k-sensitivity experiment (Section 5.3) confirms: Layer 1 AUROC decays monotonically from 0.87 (k=1) to 0.39 (k=20).

4.2 Layer 1B: Type-Restricted Distributional Score

Layer 1 uses *unrestricted* nearest neighbours: the target's closest requirement to x may belong to a different type. This lets cross-type neighbours mask type-level gaps (Figure 2, left vs. centre).

Layer 1B restricts the search to same-type requirements, operating directly at the type level.

Definition 4 (Type-restricted score)**.** *For each type t, let $\mathbf{X}^t = \{x \in \mathbf{X} : h(x) = t\}$ be reference points of type t. Define the type-restricted mean distance:*

$$d_t(P) = \frac{1}{|\mathbf{X}^t|} \sum_{x \in \mathbf{X}^t} \min_{y \in P^t} d_{\cos}(x, y), \tag{7}$$

where $P^t = \{y \in P : h(y) = t\}$. Then z-score against per-project baselines:

$$\boxed{\Psi_B[t] = \mathrm{clip}\left(\frac{d_t(P_*) - \bar{d}_t}{\hat{\sigma}_{d_t} + \varepsilon},\ -5,\ 5\right),} \tag{8}$$

where $\bar{d}_t = \frac{1}{M}\sum_j d_t(P_j)$ and $\hat{\sigma}_{d_t} = \mathrm{std}_j\{d_t(P_j)\}$.

Key properties. (i) Type-restricted 1-NN amplifies the partial-removal signal by $\sim$1.9$\times$ vs. unrestricted (verified empirically: removing 75% of a type's requirements shifts unrestricted mean φ by +0.025 but type-restricted by +0.046). (ii) Because d_t averages over all $|\mathbf{X}^t|$ reference points (typically 20–190), the type-level statistic is robust to k: AUROC is nearly flat from k=1 to k=20.

4.3 Layer 1C: Soft-Count Deficit

The baseline experiments (Section 5.7) revealed that simply *counting* requirements per type, with ground-truth labels, achieves 0.933 AUROC—proving that population statistics carry powerful signal the geometric layers ignore. The geometric score is binary ("is there *any* requirement nearby?"); the count is proportional ("how *many*?").

Layer 1C captures this count signal *without ground-truth labels* using calibrated Gibbs soft assignment.

Definition 5 (Soft type count)**.**

$$\tilde{n}_t(P) = \sum_{y \in P} p_t(y), \qquad p_t(y) = \frac{\exp\big(-d_{\cos}(y, \boldsymbol{\mu}_t)/T\big)}{\sum_{t'} \exp\big(-d_{\cos}(y, \boldsymbol{\mu}_{t'})/T\big)}, \tag{9}$$

where T is calibrated so $\mathbb{E}_y[\max_t p_t(y)]$ equals the hard-assignment accuracy (0.783), yielding $T^* = 0.021$.

Definition 6 (Soft-count gap score)**.**

$$\boxed{\Psi_{\text{count}}[t] = \text{clip}\left(\frac{\bar{\tilde{n}}_t - \tilde{n}_t(P_*)}{\hat{\sigma}_{\tilde{n}_t} + \varepsilon},\ -5,\ 5\right)} \tag{10}$$

Key property. The count signal is *linear* in the removal fraction: removing 50% of a type's requirements halves $\tilde{n}_t$, producing proportional z-score change. Empirically, the soft count has 5.3$\times$ more dynamic range than the geometric Layer 1 score across removal fractions $f \in \{1.0, 0.75, 0.50\}$.

4.4 Type-Level Fusion

Definition 7 (Unified type-level gap score)**.**

$$\boxed{\Psi[t] = (1-\gamma)\left[\,\beta\,\Psi_{\text{unr}}[t] + (1-\beta)\,\Psi_B[t]\,\right] + \gamma\,\Psi_{\text{count}}[t]} \tag{11}$$

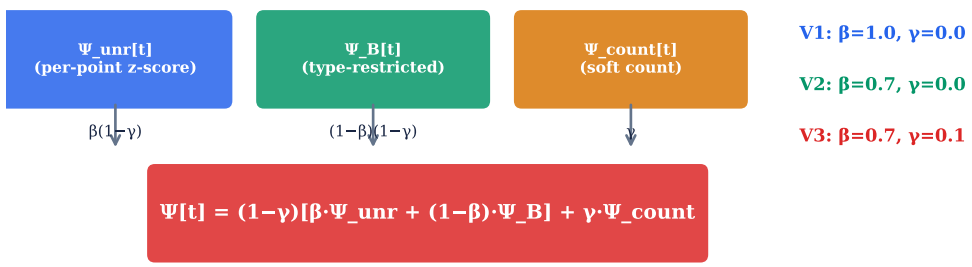

Figure 4: Type-level fusion. Three layer outputs combine via two hyperparameters (β, γ). Setting β=1, γ=0 recovers V1; β=0.7, γ=0 gives V2; β=0.7, γ=0.1 gives V3 (recommended).

4.5 Cell Aggregation onto the Interpretive Grid

Per-point ψ scores are aggregated onto a (type $\times$ topic) grid for human consumption. Each reference point x has a hard category $h(x) \in \{1, \ldots, K_t\}$ and a soft topic distribution $\boldsymbol{\pi}(x) \in \Delta^{K_s-1}$:

$$\Psi_{\text{cell}}[t, s] = \frac{\sum_{x:\, h(x)=t} \pi_s(x) \cdot \psi(x)}{\sum_{x:\, h(x)=t} \pi_s(x)}. \tag{12}$$

The **occupancy matrix** measures how much of the target falls in each cell:

$$\widetilde{M}_*[t, s] = \sum_{y \in P_*:\, h(y)=t} \pi_s(y), \qquad \textstyle\sum_{t,s} \widetilde{M}_*[t, s] = N_*. \tag{13}$$

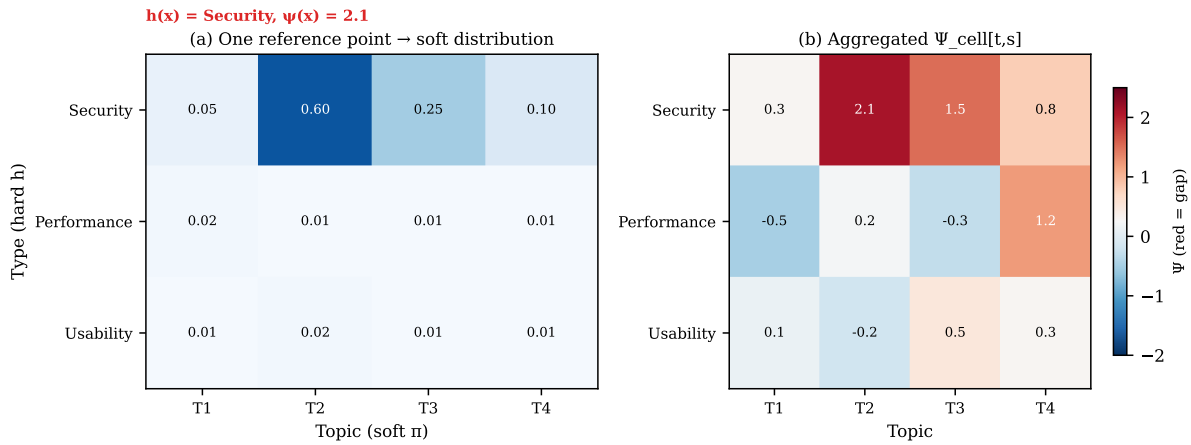

Figure 5: Soft cell aggregation. (a) One reference point with $h(x) =$ Security and $\psi = 2.1$ distributes its weight across topic columns via $\boldsymbol{\pi}$. (b) The aggregated $\Psi_{\text{cell}}[t, s]$ grid: red cells indicate gaps, blue cells indicate surplus.

Interpretive caveat. Category assignment has 78.3% accuracy and $\eta^2 \approx 0.21$ (types explain 21% of ψ variance). The grid is an approximate summary; a practitioner should read cell-level gaps as "somewhere in the neighbourhood of this type-topic combination."

4.6 Dual-Mode Comparison

The same $\varphi(x; \mathbf{Y}_j)$ values support two baselines: **Mode A** (uniform weights, RQ1): $\varphi_0^A = \frac{1}{M} \sum_j \varphi_j$. **Mode B** (similarity-weighted, RQ2): $\varphi_0^B = \sum_j w_j \varphi_j$ with $w_j \propto \exp(-d_{\cos}(\boldsymbol{\mu}_*, \boldsymbol{\mu}_j)/\tau)$. Alternative distance functions include entropic optimal transport [3].

5 Experiments

Evaluation challenge. We lack ground-truth gap labels (nobody has annotated "PROMISE_5 is missing Portability requirements"). All experiments therefore use **synthetic gap injection**: remove requirements from well-covered types, then check if the detector notices.

Protocol. Leave-One-Out Cross-Validation (LOO-CV) over all 15 projects: each takes a turn as target with the remaining 14 as training. This ensures the target never contributes to its own baseline.

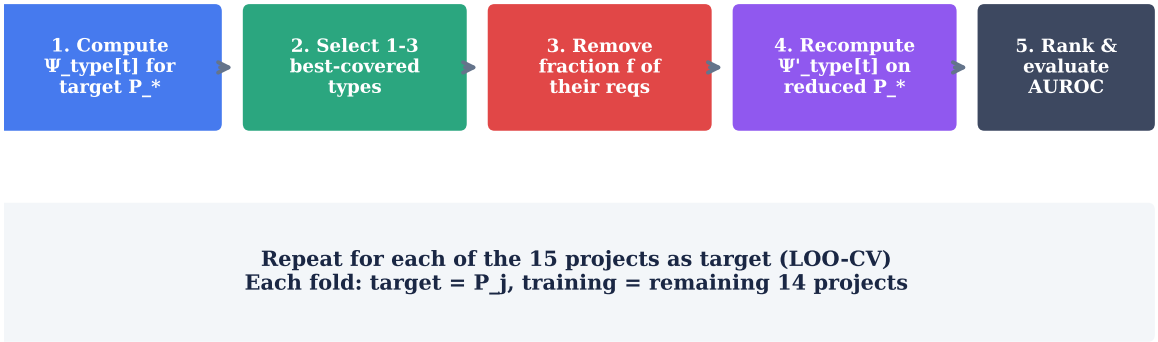

Figure 6: Synthetic gap injection protocol. For each LOO fold: (1) score all types; (2) select best-covered types; (3) remove fraction f; (4) re-score; (5) evaluate ranking.

5.1 Experiment 1: Type-Level Gap Detection

Question. Can the detector identify which requirement types are missing from a project?

Protocol. For each fold: compute $\Psi_{\text{type}}[t]$; select 1–3 best-covered types; remove all their requirements ($f = 1.0$); recompute and rank; measure AUROC, MRR, Recall@5.

Table 1: Type-level gap injection (f=1.0, k=1, corpus mode). V3 achieves 0.935 AUROC for $N \geq 50$.

	AUROC ($N \geq 50$)	**AUROC** (All)	**MRR** (All)
V1 (β=1.0, γ=0.0)	0.871 ± 0.22	0.764 ± 0.26	0.453
V2 (β=0.7, γ=0.0)	0.902 ± 0.15	0.795 ± 0.21	0.422
V3 (β=0.7, γ=0.1)	$\mathbf{0.935 \pm 0.14}$	$\mathbf{0.801 \pm 0.20}$	**0.472**

Conclusion. V3 improves over V1 by +0.064 AUROC for large projects. The improvement is concentrated on V1's weakest projects (PROMISE_8: +0.148; PROMISE_4: +0.200; PROMISE_7: +0.300).

5.2 Experiment 2: Fraction Monotonicity

Question. Does the gap score increase with gap severity?

Protocol. Same as Exp. 1 with $f \in \{1.0, 0.75, 0.50\}$.

Conclusion. Monotonicity holds, but detection degrades sharply for partial gaps. In d=1024, removing 25% of a type's requirements barely shifts the 1-NN geometry because surviving requirements still cover most reference points.

5.3 Experiment 3: k-Sensitivity

Question. Is the gap signal local (1-NN) or distributed? Is V3 robust to the choice of k?

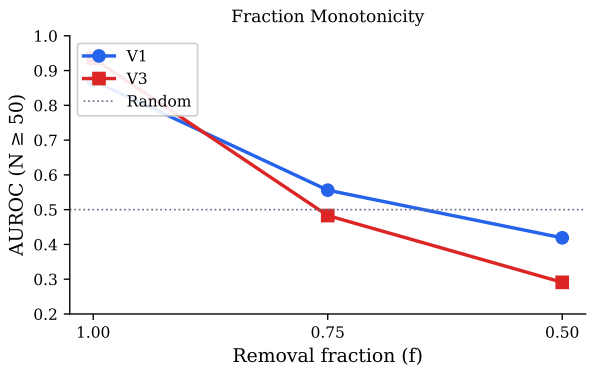

Figure 7: AUROC vs. removal fraction ($N \geq 50$). Both V1 and V3 satisfy monotonicity. The steep drop from f=1.0 to f=0.75 confirms the method is primarily a *complete-absence* detector.

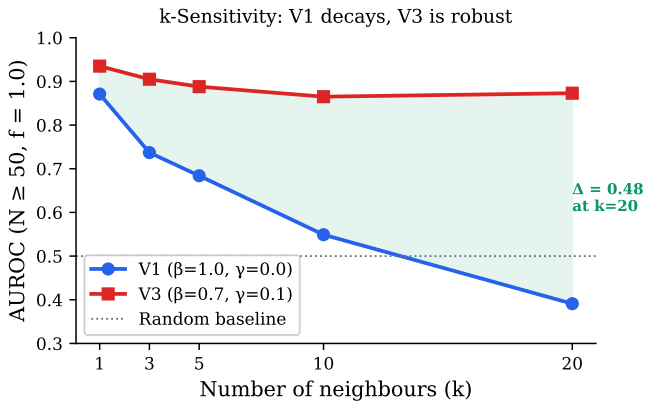

Figure 8: k-sensitivity ($N \geq 50$, f=1.0). V1 decays from 0.871 to 0.391 (strictly local signal). V3 stays above 0.87 at all k (robust). A random detector would be flat at 0.5.

Conclusion. V1 requires k=1 exactly; V3 works at any $k \in [1, 20]$. The monotonic V1 decay is also the strongest evidence the method captures a real signal (random detectors produce flat lines). V3's robustness arises because Layers 1B and 1C aggregate across all reference points per type *before* the z-score.

5.4 Experiment 4: Permutation Test

Question. Are individual-project AUROCs statistically significant?

Protocol. 1000 permutations per project: randomly label 3 types as "targets" without removal. Null 95th percentile $\approx$ 0.83.

Result. Under V3, 9 of 15 projects exceed the null threshold (all 6 large plus 3 small), vs. 8 of 15 for V1.

5.5 Experiment 5: Cell-Level Localisation

Question. Can gaps be localised to specific (type $\times$ topic) cells?

Result. AUROC = 0.51 (random). 621 points across $\sim$96 cells is too sparse. This bounds the method's resolution: type-level detection works; cell-level does not at this corpus size.

5.6 Experiment 6: Whole-Project Holdout

Question. If a project is removed from training, do its signature types appear as gaps?

Result. 84 valid pairs; AUROC = 0.50 (random). Removing 1 of 15 projects shifts the baseline by ∼7%, insufficient for detection. A larger, more diverse corpus is needed.

5.7 Experiment 7: Baseline Comparison

Motivation. Without baselines, a reviewer can argue the entire pipeline is unnecessary. We evaluate six baselines on the *identical* protocol to validate each component.

Table 2: Baseline comparison (f=1.0, k=1). Each baseline ablates one pipeline component.

Baseline	**Ablates**	**AUROC** ($N \geq 50$)	**AUROC** (All)
N1: Random	(null)	0.345	0.309
N2: Count deficit (GT)	Embeddings (needs labels)	<u>0.933</u>	0.795
N3: TF-IDF + k-NN	Neural encoder	0.615	0.549
L1: Classifier cov. [6]	k-NN geometry	0.673	0.573
L2: MMD per-type [4]	Per-project normalisation	0.265	0.230
L3: Centroid distance	Per-project normalisation	0.284	0.266
V1 (ours)	—	0.871	0.764
V3 (ours, recommended)	—	**0.935**	**0.801**

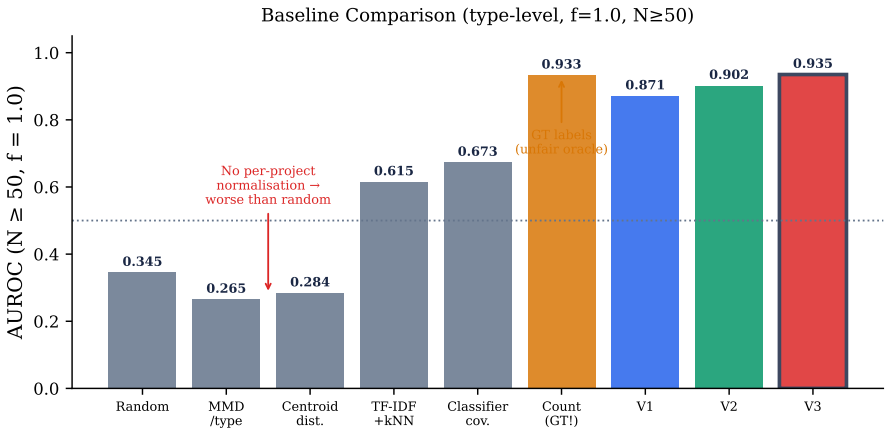

Figure 9: Baseline comparison ($N \geq 50$, f=1.0). L2 and L3 (no per-project normalisation) score *below random*. V3 matches the GT-count oracle (N2) without annotation.

Conclusions. Four validated claims:

1. **Per-project normalisation is essential.** L2 (0.265) and L3 (0.284) score below random (0.345). Without the baseline $\varphi_0(x)$, raw distances cannot distinguish genuine gaps from inherently sparse regions.
2. **Neural embeddings outperform bag-of-words.** V1 (0.871) vs. N3 (0.615): +42% relative improvement, same pipeline architecture.
3. **k-NN geometry beats classify-and-count.** V1 (0.871) vs. L1 (0.673). Continuous embedding space avoids the 22% classification error that propagates into the coverage estimate.

4. **V3 matches the GT-count oracle.** V3 (0.935) vs. N2 (0.933) for $N \geq 50$—*without human annotation.* The 0.002 gap demonstrates Layer 1C captures the population signal that GT counting exploits.

5.8 Evaluation Summary

Table 3: Consolidated findings across all seven experiments.

Experiment	AUROC	Verdict	Key finding
Type-level, V3, $N \geq 50$	0.935	Strong	Matches GT oracle
Type-level, V3, all	0.801	Good	Small projects degrade
Fraction f=0.75	0.48	Weak	Partial gaps remain hard
k-sensitivity (V3)	0.94→0.87	Robust	V1 decays to 0.39
Permutation test	—	9/15 sig	All large projects pass
Cell-level	0.51	Random	Grid too sparse
Holdout	0.50	Random	Needs larger corpus
Baselines: L2/L3	0.27	<Rand	Per-project norm essential
V3 vs. N2 (GT count)	0.935 vs 0.933	Match	No annotation needed

6 Inference on New Projects

All corpus artifacts (embeddings, centroids, topic model) are pre-computed once. For a new project P_*: (1) embed requirements with the same encoder; (2) assign soft topics via saved BERTopic; (3) compute all three gap layers against pre-computed baselines; (4) aggregate onto the grid; (5) generate a ranked report with example requirements.

Computational cost: <2 seconds on a single CPU ($N_{\text{ref}} = 621$, $M = 15$). Minimum viable $N_* \geq 10$; reliable detection requires $N_* \geq 50$.

7 Limitations

1. **Complete absences only.** AUROC 0.94 at f=1.0 but ∼0.50 at $f \leq 0.75$.
2. **Category accuracy (78.3%).** Structured misassignment can create phantom gaps.
3. **Cell-level resolution.** 621 points across 96 cells is too sparse for cell-level detection.
4. **Corpus dependency.** Gaps are relative to the corpus; uncovered concerns cannot be flagged.
5. **No practitioner validation.** Experiments test the detector's own gap definition.
6. **Statistical power.** M=14 training projects per fold limits baseline stability.

References

[1] Alibaba Cloud. Qwen3-embedding. `https://huggingface.co/Qwen/Qwen3-Embedding-0.6B`, 2025.

[2] Jane Cleland-Huang, Raffaella Settimi, Xuchang Zou, and Peter Solc. The detection and classification of non-functional requirements with application to early aspects. In *14th IEEE International Requirements Engineering Conference (RE'06)*, pages 39–48. IEEE, 2006.

[3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems 26 (NeurIPS)*, pages 2292–2300, 2013.

[4] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773, 2012.

[5] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[6] Tobias Hey, Jan Keim, Anne Koziolek, and Walter F. Tichy. NoRBERT: Transfer learning for requirements classification. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, pages 169–179. IEEE, 2020.

[7] Michel Ledoux. *The Concentration of Measure Phenomenon*, volume 89 of *Mathematical Surveys and Monographs*. American Mathematical Society, 2001.

[8] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017.

[9] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.